# Dependency Injection

DI · Wire Greeter without hard `new` in controllers · Build with Waqar

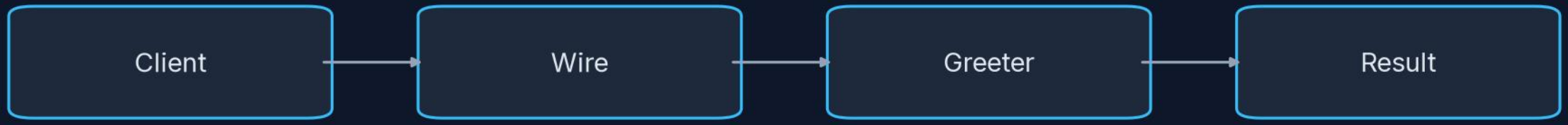
# Today's Hook

- Problem: Wire Greeter without hard `new` in controllers
- Pattern: DI — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

# The Problem

- Wire Greeter without hard `new` in controllers
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

# Structure




DI: Wire Greeter without hard `new` in controllers

# Real-World Scenario

- Running example: Wire Greeter without hard `new` in controllers
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

# Naive Code

```
package com.buildwithwaqar.patterns.enterprise.di.problem;

import java.util.*;

public final class NaiveController {
  public String hello(String name){ return new GreeterHard().greet(name); }
}
final class GreeterHard { String greet(String n){ return "hi "+n; } }
```

# Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

# Solution

```
package com.buildwithwaqar.patterns.enterprise.di.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.concurrent.locks.*;
import java.util.function.*;

interface Greeter { String greet(String name); }
final class FormalGreeter implements Greeter { public String greet(String n){ return "Hello, "+n; } }
final class DiContainer {
  private final Map<Class<?>, Object> beans=new HashMap<>();
  <T> void register(Class<T> type, T impl){ beans.put(type, impl); }
  @SuppressWarnings("unchecked") <T> T get(Class<T> type){
    Object o=beans.get(type); if(o==null) throw new IllegalStateException("missing "+type.getSimpleName()); return (T)o;
  }
}
final class HelloController {
  private final Greeter greeter; HelloController(Greeter g){ greeter=g; }
  String hello(String name){ return greeter.greet(name); }
}
```

# Runnable main

```
package com.buildwithwaqar.patterns.enterprise.di.problem;
import java.util.*; import java.util.concurrent.*;
public final class ProblemDemo {
  public static void main(String[] args) throws Exception {
    System.out.println("=== DI: wire Greeter without hard new ===");
    System.out.println("--- naive ---"); System.out.println(new NaiveController().hello("Ada"));
System.out.println("--- DI ---");
var c=new DiContainer(); c.register(Greeter.class, new FormalGreeter());
var ctl=new HelloController(c.get(Greeter.class));
System.out.println(ctl.hello("Ada"));
try { new DiContainer().get(Greeter.class); } catch (IllegalStateException e){ System.out.println("caught="+e.getMessage()); }
  }
}
```

# Walkthrough Output

```
=== DI: wire Greeter without hard new ===
--- naive ---
hi Ada
--- DI ---
Hello, Ada
caught=missing Greeter
```

# Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

# Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

# Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with DI.

# Related Patterns

- Contrast DI with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

# Recap

- Problem → naive → DI → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.